6.1 Report

Introduction

LLMOps is the practice of building, deploying, and operating applications powered by large language models. It extends MLOps, which manages data, training, validation, deployment, and monitoring, to handle generative systems' additional operational needs. Teams version prompts, retrieval context, guardrails, and output-quality criteria alongside model and code. They also monitor inference cost, latency, and token use, which vary with the input and generated response. These concerns interact: changing a prompt can affect correctness, format, latency, and cost even when the model stays fixed.

Latency Findings

The local `llama3.2:3b` model completed all five baseline requests successfully. Latency averaged 11,476 ms (11.48 seconds), with a median of 10,255 ms, standard deviation of 3,947 ms, minimum of 7,888 ms, and maximum of 15,847 ms. The slowest request was question 4 about the water cycle and its main stages. In the separate token-count and cost pass, it also had the longest response at 434 output tokens, consistent with generation length contributing to delay. Because latency and token counts came from separate calls, this is suggestive rather than causal evidence. Hardware load and inference variability also matter. Five requests are too few to characterize production tail latency.

Cost Analysis

Because the model ran locally, there was no per-token provider invoice. The notebook estimates compute cost using an assumed rate of $0.10 per hour multiplied by each request's elapsed inference time. The average is $0.00031537 per request; at 10,000 requests per month, projected compute cost is $3.1537. This proxy is not total cost of ownership: it excludes electricity, idle capacity, hardware purchase or depreciation, and operational overhead, and the hourly rate may not match deployment conditions. To optimize, cap output length, request concise answers, and use a smaller model for simple questions when quality remains acceptable. Shorter prompts and caching repeated context may also reduce processing time and token use.

Prompt Evaluation

Three registered prompt versions were tested on the same five academic questions. Version 1 simply asked for a clear answer. Version 2 added an academic-assistant role and requested concise, accurate, undergraduate-friendly language. Version 3 required JSON containing `answer`, `confidence`, and `key_points`. Figures 1–3 document the prompt comparisons and v3 template; Figure 4 shows the logged evaluation metrics, and Figure 5 confirms the production alias. The `valid_json` check parsed each v3 response and verified all required keys. It passed for all five: 5/5, or 100%. This metric was recorded only for v3, so it does not compare JSON performance across versions or measure factual accuracy, completeness, or confidence calibration. The notebook assigned the production alias to v3. This is reasonable when downstream systems require structured JSON, but should be considered an initial promotion: the sample is small and the metric tests format, not answer quality. A broader evaluation should include semantic-quality checks and failure cases before high-impact use.

Conclusion

Operationalizing an LLM application requires more than making inference work. Track prompt versions and output contracts, and evaluate both machine-checkable behavior and answer quality. Latency and token data expose performance and cost trade-offs, while local-model estimates need explicit assumptions to avoid implying full deployment cost. Production promotion should follow an evaluation gate rather than a hard-coded choice. More requests, hardware conditions, and representative questions would make the decision more robust.

MLflow Screenshots

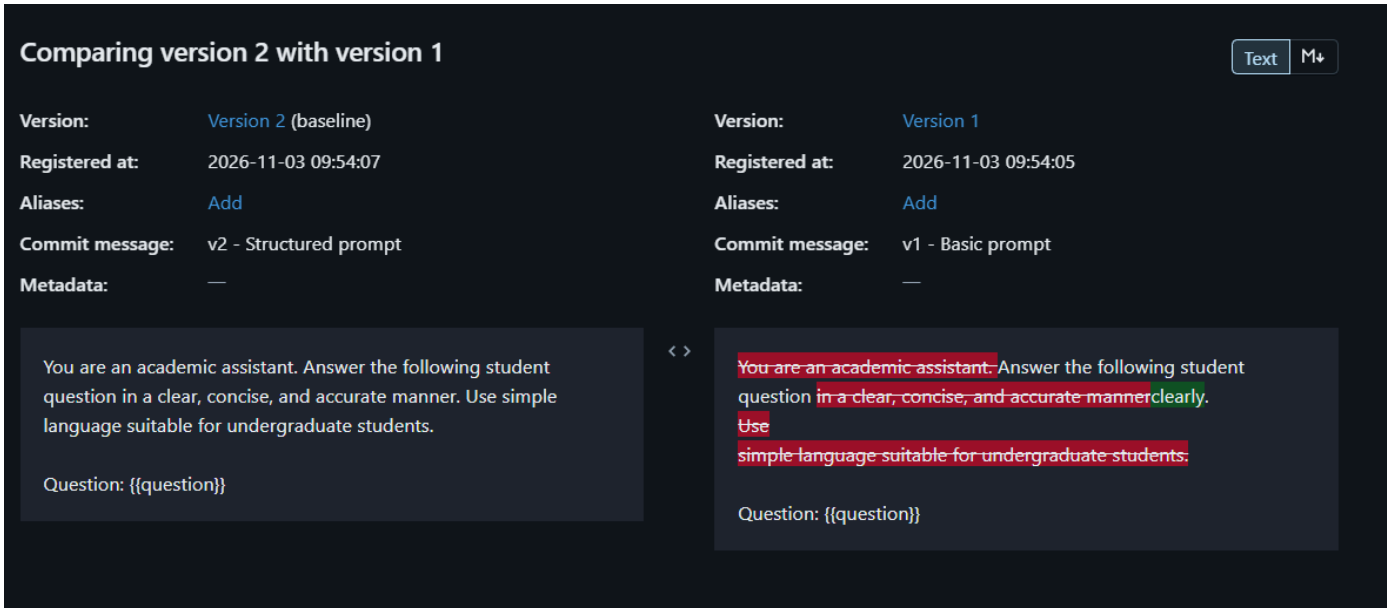


Figure 1. Comparison of prompt versions 2 and 1.

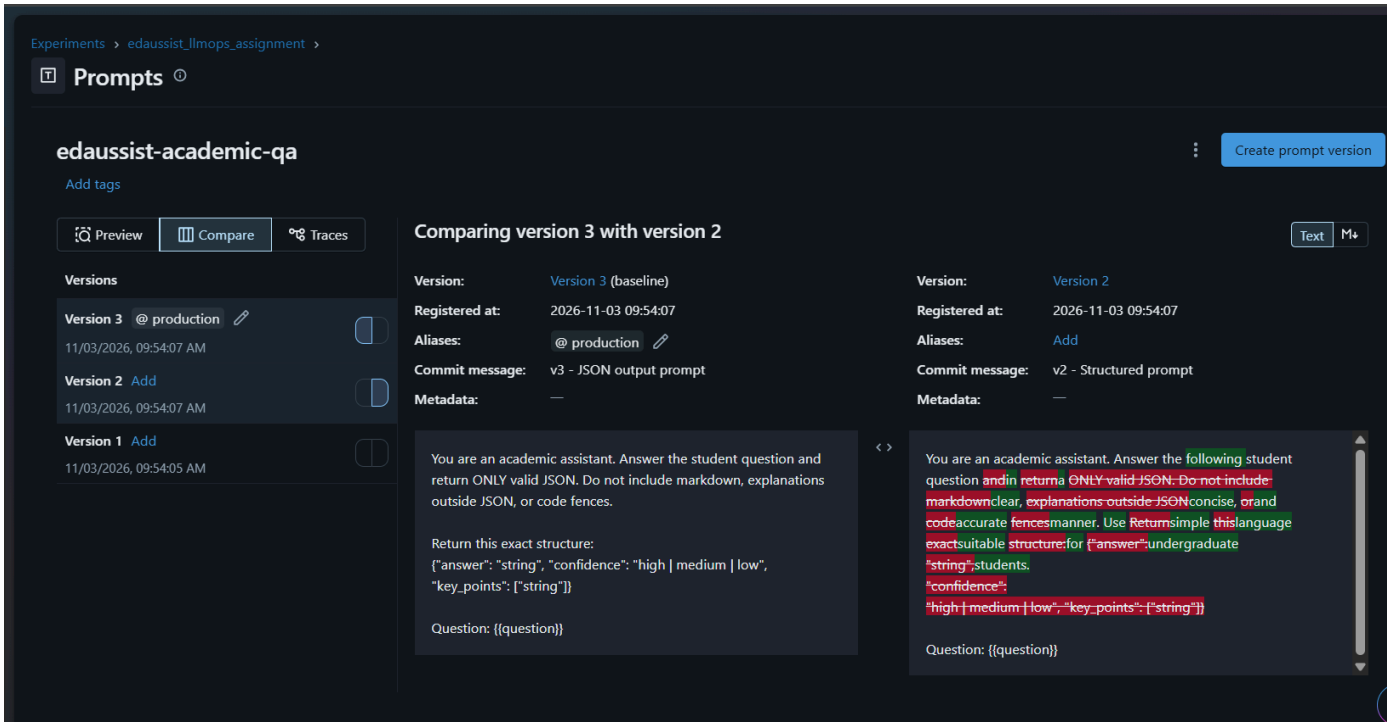


Figure 2. Comparison of prompt versions 3 and 2; v3 is production.

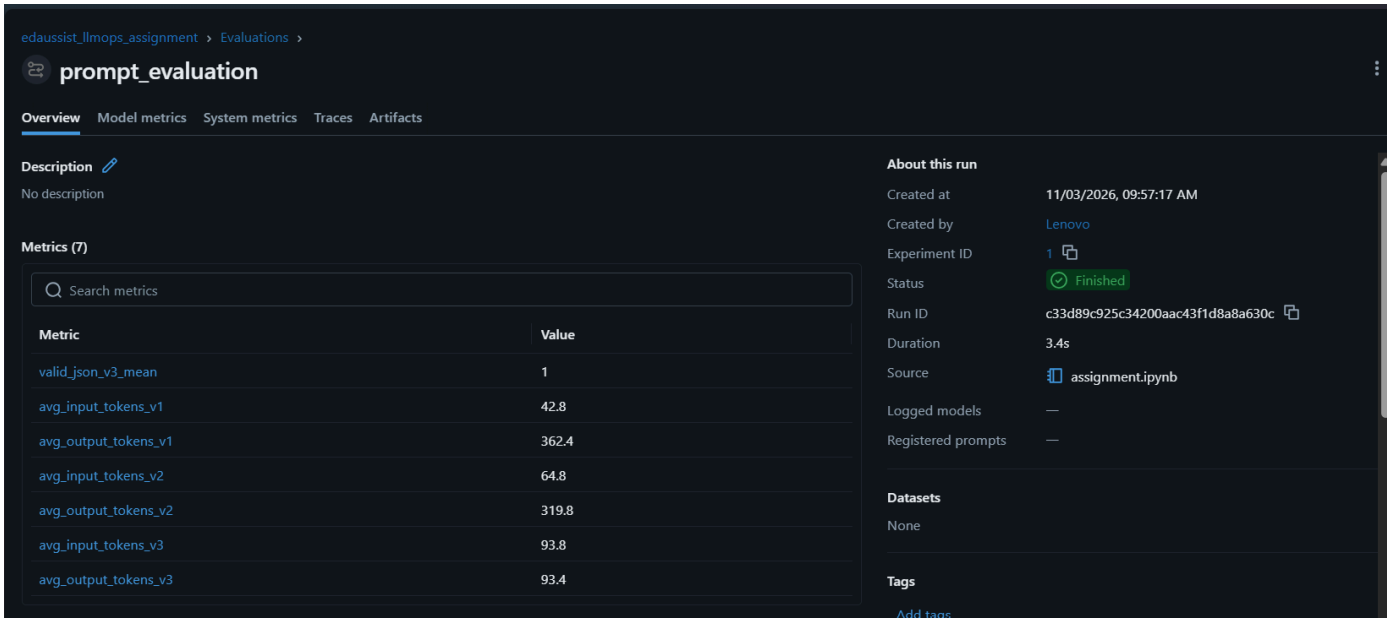


Figure 3. Version 3 JSON prompt preview.

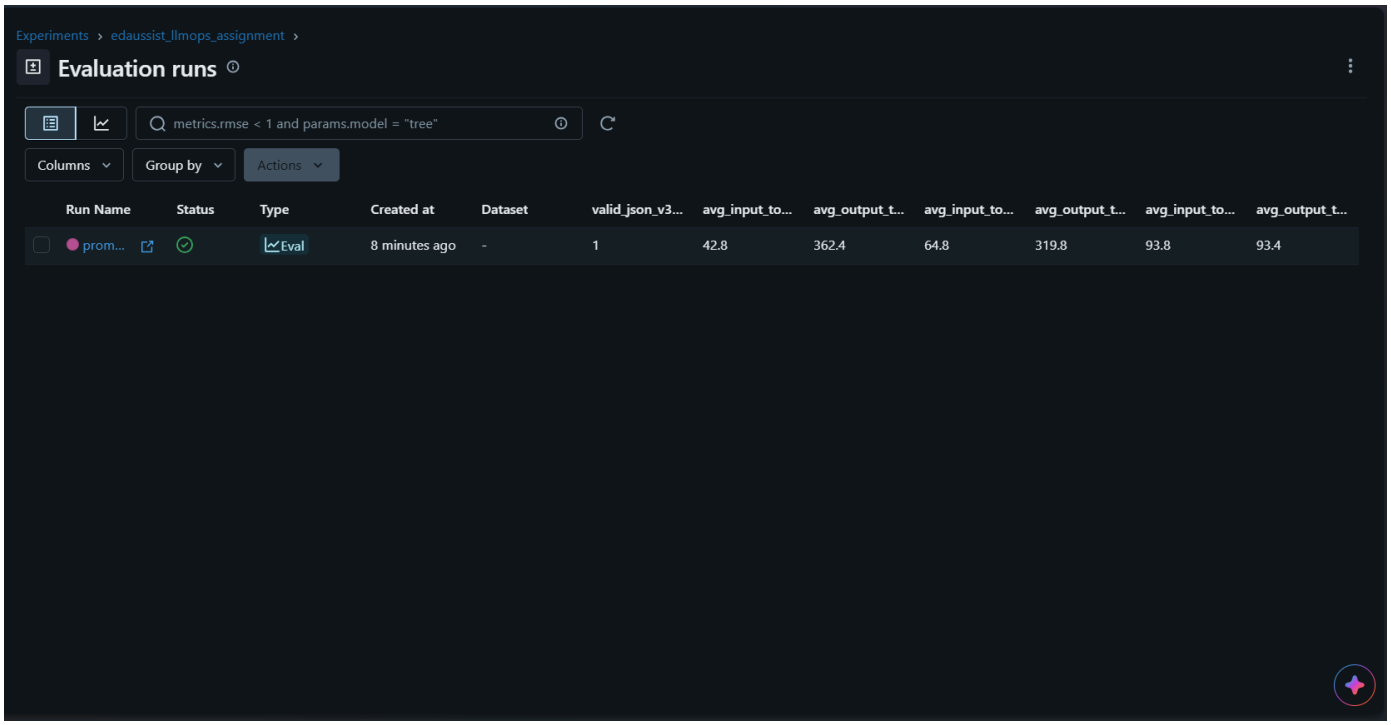


Figure 4. MLflow evaluation metrics.

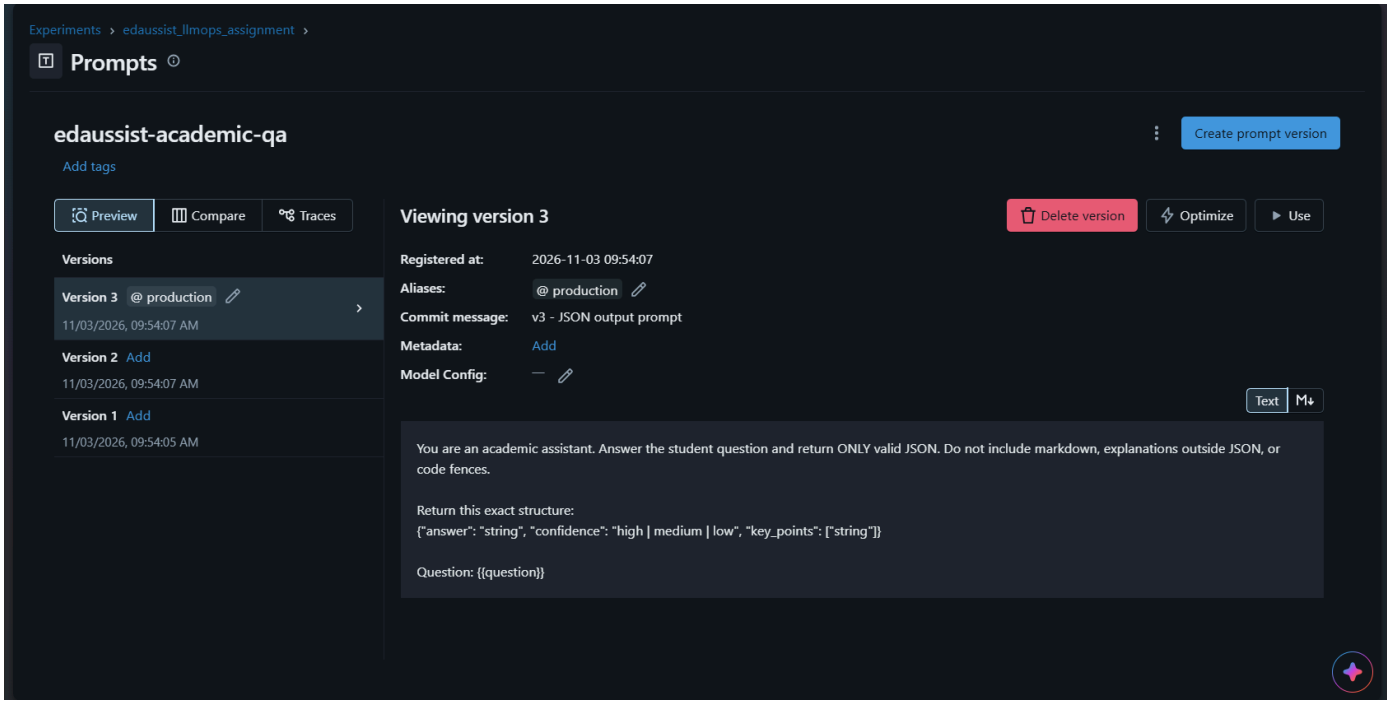


Figure 5. Production alias assigned to version 3.